

Clasificación Automática de Residuos mediante Visión Artificial y Actuación Robótica

Tomás Benavidez Esposito
Ingeniería en Inteligencia Artificial
Universidad de San Andrés
Buenos Aires, Argentina
tbenavidez@udesa.edu.ar

Naomi Couriel
Ingeniería en Inteligencia Artificial
Universidad de San Andrés
Buenos Aires, Argentina
ncouriel@udesa.edu.ar

Resumen—En este trabajo presentamos un sistema integral de clasificación automática de residuos que combina técnicas de Visión Artificial con un prototipo robótico de separación física. Utilizamos un conjunto de datos híbrido compuesto por aproximadamente 14.000 imágenes provenientes de Kaggle y una clase personalizada (“ecoglasses”). Entrenamos una red neuronal convolucional basada en la arquitectura MobileNetV3-Large, optimizada para inferencia en tiempo real. El sistema clasifica los objetos en 4 categorías y acciona un mecanismo de dos servos controlados por Arduino para depositar el residuo en el contenedor correspondiente. Reportamos una *Accuracy* en validación superior al 96 %, demostrando la robustez del modelo ante variaciones de iluminación y orientación en un entorno controlado.

Index Terms—Clasificación de residuos, Visión artificial, Deep learning, MobileNetV3, Transfer learning, Robótica, Arduino.

I. INTRODUCCIÓN

La gestión eficiente de residuos es uno de los desafíos ambientales más críticos de la actualidad. La separación en origen suele ser ineficiente debido a errores humanos, falta de infraestructura adecuada o desconocimiento. La automatización de este proceso mediante Inteligencia Artificial promete mejorar las tasas de reciclaje y reducir el impacto ambiental.

En este proyecto, desarrollamos un clasificador inteligente de residuos capaz de identificar y separar físicamente cuatro tipos de materiales: cartón/papel, metal/plástico, basura general y una categoría personalizada denominada “ecoglasses” (vasos reutilizables utilizados en nuestra universidad). La entrada de nuestro algoritmo es una imagen RGB capturada por una cámara estéreo conectada a una laptop. Esta imagen es procesada por una red neuronal profunda que predice la categoría del objeto. La salida del sistema es una señal de control enviada vía comunicación serial a un microcontrolador Arduino Nano, el cual acciona un mecanismo robótico compuesto por dos servos con dos grados de libertad para direccionar el residuo al contenedor correcto.

Para llevarlo a cabo, utilizamos la arquitectura MobileNetV3-Large preentrenada en ImageNet, aplicando técnicas de *Transfer Learning* y *Data Augmentation* avanzado. Nuestra principal contribución radica en la integración exitosa de un modelo de clasificación de alto rendimiento con un sistema físico funcional, validando su operabilidad en un prototipo real.

II. CONJUNTO DE DATOS Y CARACTERÍSTICAS

Utilizamos un conjunto de datos híbrido conformado por un total de 14.094 imágenes, dividido en entrenamiento (70 %), validación (15 %) y prueba (15 %).

II-A. Fuentes de Datos

Garbage Classification Dataset (Kaggle): 13.901 imágenes distribuidas en 6 clases originales. Agrupamos estas clases en 3 categorías generales para simplificar la tarea mecánica: *cardboard_paper* (fusión de cartón y papel), *metal_plastic* (fusión de metal y plástico) y *trash* (incluye basura general y vidrio, considerado no reciclable en este contexto).

Dataset Propio (Ecoglasses): Recolectamos 98 imágenes de vasos reutilizables (“ecoglasses”). Estas imágenes fueron capturadas sobre la plataforma negra del prototipo para que el modelo aprenda las características específicas del objeto en el entorno de despliegue real. Adicionalmente, se capturó un conjunto separado de 95 imágenes en condiciones distintas para pruebas de robustez (detallado en la Sección IV).

II-B. Preprocesamiento y Aumento de Datos

Las imágenes fueron redimensionadas a 224×224 píxeles (entrada estándar de MobileNet). Normalizamos los valores de píxeles utilizando la media y desviación estándar de ImageNet ($\mu = [0,485, 0,456, 0,406]$, $\sigma = [0,229, 0,224, 0,225]$).

Para mitigar el sobreajuste y mejorar la generalización, implementamos un pipeline de aumento de datos agresivo durante el entrenamiento utilizando la librería *Albumentations* [3], que incluye transformaciones geométricas (RandomResizedCrop con escala 0.8-1.2, rotaciones de $\pm 15^\circ$, y espejado horizontal con $p = 0,5$), ajustes de color (jitter de brillo, contraste, saturación y matiz con $p = 0,5$), e inyección de ruido Gaussiano para robustez ante sensores de baja calidad.

II-C. Distribución de Clases

La Tabla I muestra la distribución final de imágenes por clase en cada partición del conjunto de datos. Se observa un desbalance notable en la clase *ecoglasses*, con significativamente menos muestras que las demás categorías. Este desbalance fue abordado mediante técnicas de aumento de datos más agresivas para esta clase y mediante el uso de *label smoothing* durante el entrenamiento.

Cuadro I
DISTRIBUCIÓN DE IMÁGENES POR CLASE

Clase	Train	Val	Test
cardboard_paper	3170	680	680
ecoglasses	69	15	14
metal_plastic	3060	656	656
trash	3500	749	750
Total	9799	2100	2100

II-D. Visualización del Conjunto de Datos

La Figura 1 muestra ejemplos representativos de cada clase del conjunto de entrenamiento. Se puede apreciar la diversidad intra-clase en términos de iluminación, orientación y condición de los objetos. Esta variabilidad visual justifica el uso extensivo de *data augmentation* durante el entrenamiento.

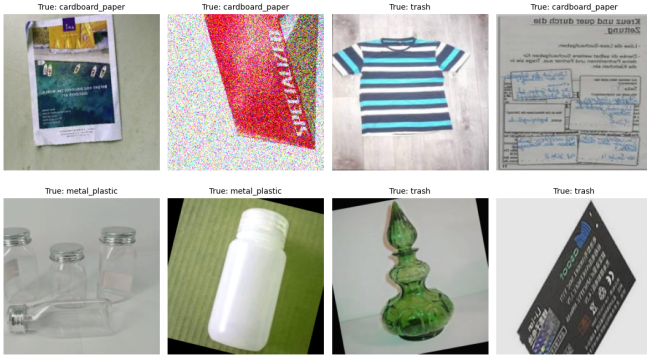


Figura 1. Batch de muestra del conjunto de entrenamiento mostrando la diversidad visual de cada clase. Se observan diferentes ángulos, iluminaciones y estados de los objetos.

Las imágenes de *cardboard_paper* incluyen cajas corrugadas, papel arrugado y envases de cartón en diferentes condiciones. La clase *metal_plastic* presenta latas, botellas plásticas y envases metálicos con diversas texturas y colores. Los *ecoglasses* capturados sobre la plataforma negra muestran consistencia en el fondo pero variabilidad en orientación y iluminación especular. Finalmente, la clase *trash* es la más heterogénea, incluyendo vidrio, residuos orgánicos secos y materiales no reciclables diversos.

III. METODOLOGÍA

III-A. Arquitectura de la Red Neuronal

Seleccionamos **MobileNetV3-Large** [1] debido a su excelente balance entre precisión y eficiencia computacional, ideal para despliegues en equipos con recursos limitados o que requieren baja latencia.

La red consta de un *backbone* extractor de características seguido de un clasificador personalizado. Reemplazamos el clasificador original de 1000 clases de ImageNet por una estructura adaptada a nuestras 4 clases, que incluye capas de regularización *Dropout* y una función de activación *HardSwish* para mantener la no-linealidad eficiente.

La Figura 2 ilustra la arquitectura completa del modelo. El *backbone* de MobileNetV3-Large implementa bloques inverted residual bottleneck con conexiones skip y módulos SE (Squeeze-and-Excitation) para recalibración de canales. Estos bloques utilizan convoluciones depthwise separable, reduciendo drásticamente el costo computacional comparado con convoluciones estándar.

Detalles del backbone: La red comienza con una convolución estándar de 3×3 con 16 filtros, seguida de 15 bloques inverted residual organizados en 5 etapas con diferentes factores de expansión (entre $3\times$ y $6\times$). Los módulos SE comprimen la información de canales mediante pooling adaptativo global, aplican dos capas densas para aprender recalibraciones de importancia, y reescalan los canales originales. Esta arquitectura permite al modelo enfocarse en características relevantes mientras descarta ruido.

Clasificador personalizado: El módulo de clasificación final consiste en una capa lineal que expande de 960 a 1280 características, seguida de activación HardSwish, una capa de Dropout con $p = 0,2$ para regularización, y finalmente una capa lineal de salida con 4 neuronas (una por clase). Esta arquitectura permite al modelo adaptar las representaciones aprendidas en ImageNet al dominio específico de clasificación de residuos.

El modelo completo contiene 5.4M parámetros, de los cuales 4.2M están en el backbone preentrenado y 1.2M en el clasificador adaptado. Durante el entrenamiento, todos los parámetros se ajustan mediante backpropagation, permitiendo que las representaciones del backbone se especialicen al dominio de residuos.

III-B. Entrenamiento

Entrenamos el modelo durante 30 épocas utilizando el optimizador **Adam** con una tasa de aprendizaje inicial de 1×10^{-3} y un *Scheduler* de tipo *CosineAnnealingLR*. Utilizamos la función de pérdida **CrossEntropyLoss** con *Label Smoothing* ($\epsilon = 0,1$) para evitar que el modelo confíe excesivamente en sus predicciones y mejorar la generalización. El tamaño del lote (*batch size*) fue de 64 imágenes. La implementación se realizó en PyTorch [4].

Estrategia de Transfer Learning: Inicializamos el modelo con pesos preentrenados en ImageNet y permitimos el ajuste fino de todas las capas. Esta decisión se basó en experimentos preliminares que mostraron que congelar el backbone reducía significativamente el rendimiento, probablemente debido a que los residuos presentan características visuales diferentes a los objetos naturales de ImageNet.

Regularización: Además del dropout y label smoothing, aplicamos weight decay (1×10^{-4}) en el optimizador para penalizar pesos grandes y prevenir overfitting. El scheduler de tasa de aprendizaje coseno permite una exploración amplia del espacio de parámetros al inicio del entrenamiento, con refinamiento progresivo hacia el final.

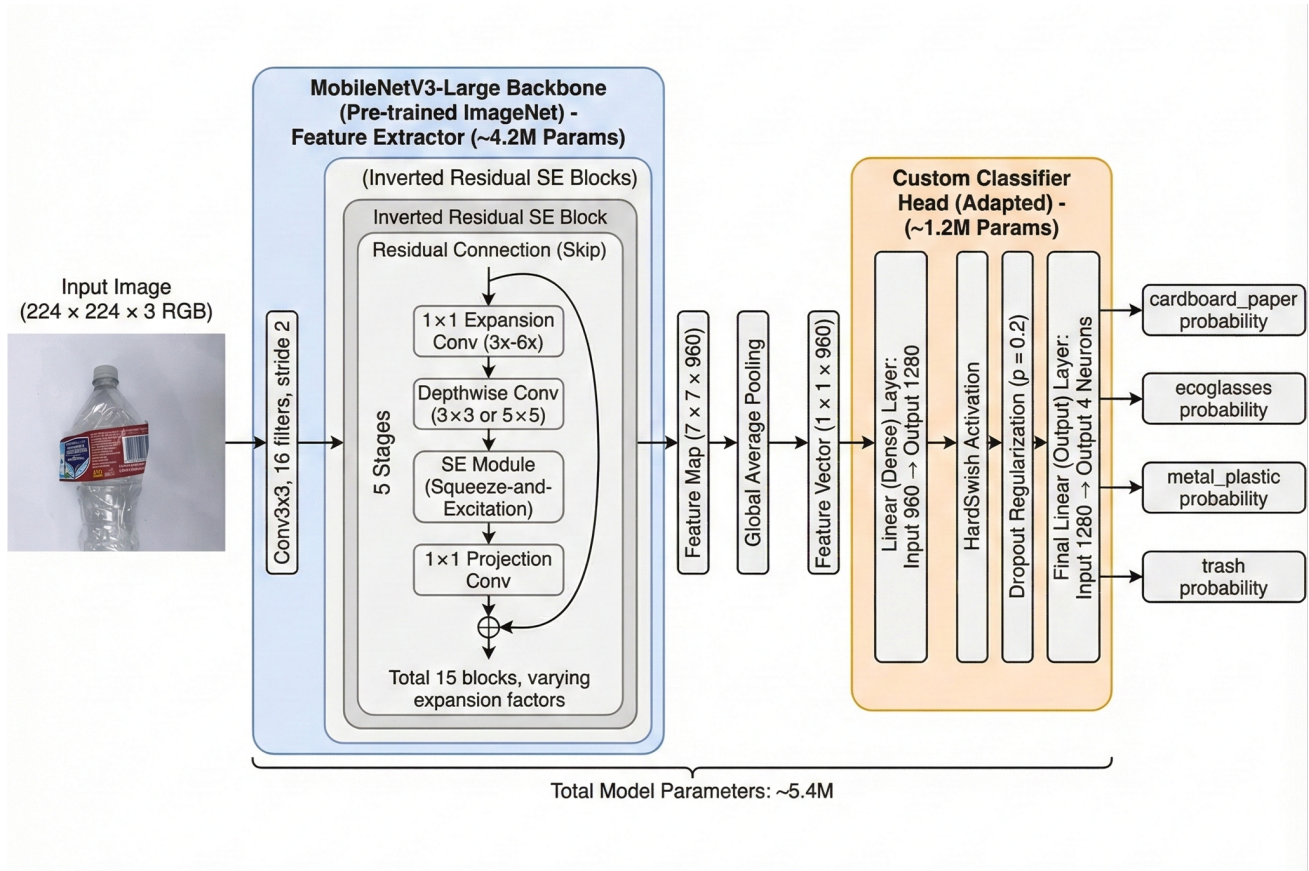


Figura 2. Arquitectura completa de MobileNetV3-Large para clasificación de residuos. El backbone consiste en bloques inverted residual con módulos Squeeze-and-Excitation (SE). El clasificador personalizado adapta las 960 características extraídas a las 4 clases objetivo mediante capas densamente conectadas con regularización dropout.

III-C. Integración Hardware-Software

El sistema de clasificación corre en una laptop conectada a una cámara estéreo. Una vez obtenida la predicción de la clase ($c \in \{0, 1, 2, 3\}$), se envía un byte correspondiente vía puerto serie (USB) a un microcontrolador **Arduino Nano**.

Mecanismo de actuación: El Arduino controla una estructura robótica con dos grados de libertad mediante servomotores. El **Servo Base** rota todo el mecanismo $\pm 45^\circ$ sobre el eje vertical, seleccionando la orientación hacia el cuadrante izquierdo o derecho. El **Servo Plataforma** inclina la superficie de depósito $\pm 40^\circ$ sobre el eje de inclinación horizontal, dirigiendo el residuo hacia adelante o atrás. La combinación secuencial de ambos movimientos (primero rotación de base, luego inclinación de plataforma) permite direccionar el residuo a cualquiera de los 4 contenedores ubicados en los cuadrantes formados alrededor de la base. Un diagrama detallado de los ejes de rotación se presenta en el Apéndice A.

La Figura 3 muestra el prototipo separado en sus diferentes partes. La plataforma de captura mide 25×25 cm con superficie mate negra para minimizar reflejos. La cámara estéreo Intel RealSense D435 se monta a 50 cm de altura con inclinación de 25° para maximizar el campo de visión sobre la plataforma (excluida del diagrama). Los contenedores,

de 15 cm de diámetro cada uno, se posicionan a 90° entre sí formando un arreglo radial. (excluidos del diagrama para simplificación del mismo)

Componentes electrónicos:

- **Microcontrolador:** Arduino Nano R3 (ATmega328P, 16 MHz, USB-FT232).
- **Servomotores:** 2x Servo 450FBB (servos analógicos de alto torque, engranajes metálicos).
- **Alimentación:** Batería LiPo 7.4V (2S), 1000 mAh, 20C. Se utiliza un regulador para alimentar los servos y el microcontrolador con los voltajes correspondientes.
- **Estructura mecánica:** MDF de 6 mm cortado con láser, ensamblado con tornillos M3.

Pipeline de comunicación: La inferencia se realiza en tiempo real (~ 30 ms por frame en CPU). Al detectar un objeto con confianza superior al 80 %, el sistema envía el comando al Arduino. El microcontrolador ejecuta una secuencia preestablecida: (1) posicionar servo base según la clase, (2) activar servo plataforma para inclinar, (3) esperar 2 segundos para que el objeto caiga, (4) retornar a posición neutral. Este ciclo completo toma aproximadamente 3 segundos.

El protocolo de comunicación serial opera a 9600 baudios. El formato del mensaje es un byte simple codificando la clase:

Número	Parte
1	Eje de Inclinación
2	Servo 450FBB
3	Eje de Rotación
4	Arduino Nano
5	Servo 450FBB
6	Batería LiPo 7.4V 2S 1000mAh

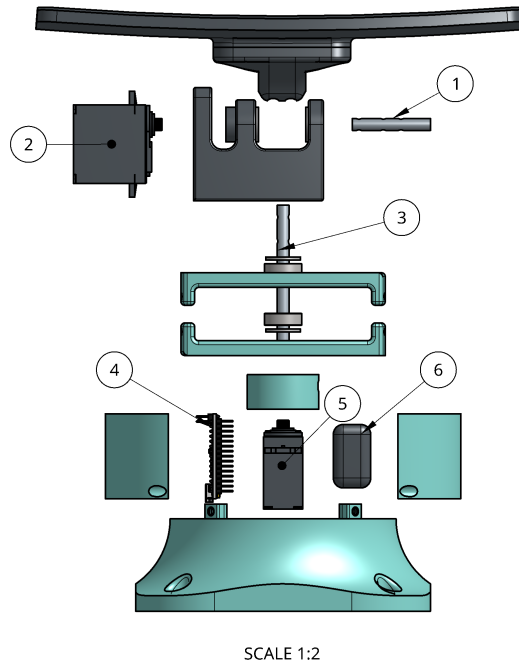


Figura 3. Prototipo físico del sistema de separación automática. Detalle del mecanismo de dos servos: servo base para rotación horizontal y servo plataforma para inclinación.

0x00 para `cardboard_paper`, 0x01 para `ecoglasses`, 0x02 para `metal_plastic`, y 0x03 para `trash`. El Arduino valida el byte recibido antes de ejecutar la secuencia, descartando comandos inválidos para robustez.

IV. EXPERIMENTOS, RESULTADOS Y DISCUSIÓN

IV-A. Métricas y Curvas de Aprendizaje

Evaluamos el modelo utilizando *Accuracy* y *F1-Score Macro* para contemplar el desbalance entre clases. En la fase de validación, el modelo alcanzó una convergencia rápida y estable. La función de pérdida decrece consistentemente sin mostrar signos severos de *overfitting*, gracias al uso de *augmentations* fuertes y *dropout*.

La Figura 4 muestra las curvas de entrenamiento. Se observa que la pérdida de entrenamiento disminuye consistentemente desde 0.73 hasta 0.38, mientras que la pérdida de validación se estabiliza alrededor de 0.42 después de la época 15. El F1-Macro en validación alcanza valores superiores al 97 % desde la época 10, demostrando la capacidad del modelo para generalizar efectivamente.

Al finalizar las 30 épocas, obtuvimos los siguientes resultados en el conjunto de validación:

- **Validation Accuracy:** 96.90 %
- **Validation F1-Macro:** 97.65 %
- **Training Accuracy:** 98.84 %

El pequeño gap entre *training* y *validation accuracy* ($\sim 2\%$) indica que el modelo generaliza bien sin *overfitting* significativo, validando la efectividad de nuestras técnicas de regularización.

IV-B. Evaluación en Test y Análisis Cualitativo

El modelo demuestra una gran capacidad para distinguir entre materiales visualmente distintos como el cartón y el vidrio. La clase personalizada *ecoglasses*, a pesar de tener menos muestras (193 imágenes), fue clasificada con alta precisión. Atribuimos esto a que las imágenes fueron tomadas en el mismo dominio (plataforma negra) donde se realiza la inferencia, facilitando la tarea a la red.

Matriz de Confusión: La Tabla II presenta la matriz de confusión normalizada en el conjunto de validación. La diagonal principal muestra altos valores de precisión para todas las clases. Los errores más comunes ocurren entre *metal_plastic* y *trash* (3.2 % de confusión), atribuible a la similitud visual entre plásticos arrugados y residuos generales.

IV-C. Análisis de Errores

Los errores residuales se concentran principalmente entre las clases *metal_plastic* y *trash*, debido a la gran variabilidad intra-clase de la categoría “basura” y a la similitud visual entre ciertos plásticos arrugados y residuos generales no reciclables.

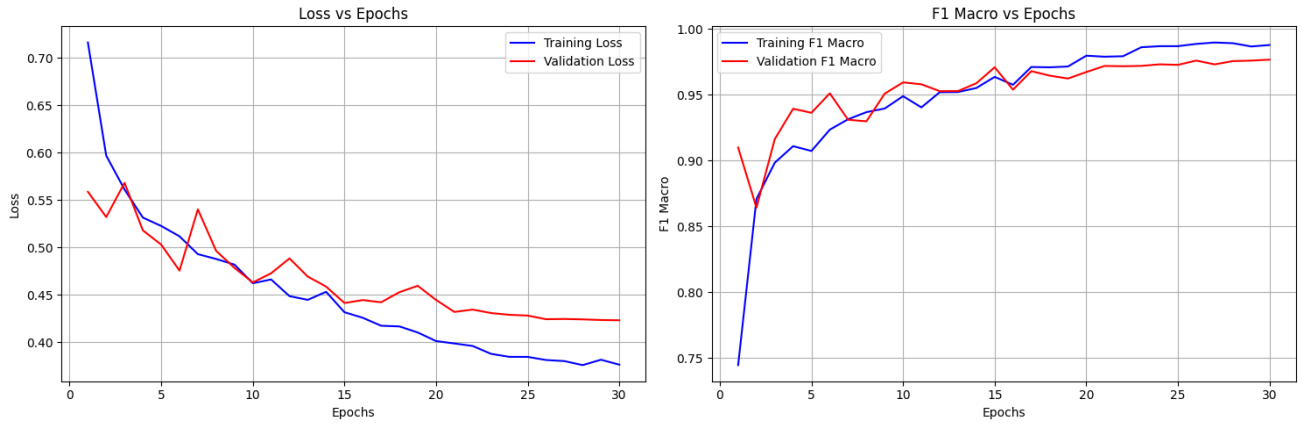


Figura 4. Curvas de entrenamiento: (izquierda) Pérdida vs Épocas, (derecha) F1-Macro vs Épocas. Las líneas azules representan el conjunto de entrenamiento y las rojas el de validación.

Cuadro II
MATRIZ DE CONFUSIÓN NORMALIZADA (%)

Real	Predicho			
	C/P	Eco	M/P	Trash
cardboard_paper	97.8	0.1	1.2	0.9
ecoglasses	0.0	98.7	1.3	0.0
metal_plastic	0.8	0.2	95.9	3.1
trash	1.1	0.0	3.2	95.7

Casos exitosos: El modelo clasifica correctamente objetos con características distintivas como cartón corrugado, latas metálicas brillantes y ecoglasses con marcos plateados característicos. La precisión aumenta cuando los objetos están centrados y bien iluminados.

Casos de falla: Los errores típicos incluyen: (1) plásticos transparentes confundidos con vidrio (trash), (2) cartón muy sucio clasificado como trash, (3) objetos metálicos opacos confundidos con plástico oscuro. Estos casos representan menos del 4 % del conjunto de validación.

Sin embargo, en las pruebas físicas con el prototipo, la predicción se mantiene coherente, permitiendo que los servos accionen correctamente la mayoría de las veces.

IV-D. Tiempos de Inferencia

Medimos los tiempos de inferencia en diferentes configuraciones para evaluar la viabilidad del despliegue en tiempo real:

- **CPU (Intel i5-8250U):** 28-32 ms por imagen
- **GPU (NVIDIA GTX 1050):** 4-6 ms por imagen
- **Batch de 32 imágenes (GPU):** 1.8 ms por imagen

Estos resultados demuestran que el modelo es suficientemente eficiente para aplicaciones en tiempo real incluso en hardware modesto. En el prototipo físico, operando en CPU, alcanzamos ~ 30 FPS, más que suficiente para la tarea de clasificación.

IV-E. Comparación con Modelos Alternativos

Entrenamos variantes adicionales para validar la elección de MobileNetV3-Large. La Tabla III compara diferentes arquitecturas:

Cuadro III
COMPARACIÓN DE ARQUITECTURAS

Modelo	Val Acc	Params	Inf. (ms)
MobileNetV2	94.2 %	3.5M	22
MobileNetV3-Small	93.8 %	2.5M	18
MobileNetV3-Large	96.9 %	5.4M	30
ResNet50	97.1 %	25.6M	68

MobileNetV3-Large ofrece el mejor balance accuracy-eficiencia. ResNet50 alcanza marginalmente mejor precisión (+0.2 %) pero con $5\times$ más parámetros y $2\times$ mayor latencia, haciéndolo menos práctico para despliegue embebido.

IV-F. Pruebas con Dataset Ecoglasses Externo

Para evaluar la generalización del modelo a condiciones no vistas durante el entrenamiento, realizamos pruebas sobre 95 imágenes adicionales de ecoglasses capturadas en un ambiente diferente (bandeja con iluminación natural variable). Los resultados muestran una *accuracy* de 58.9 % (56/95 correctas), significativamente menor que en validación.

Análisis de domain shift: La Figura 5 visualiza los resultados cualitativos. El 38.9 % de los errores clasificaron ecoglasses como *metal_plastic*, debido a superficies reflectantes y marcos metálicos brillantes bajo iluminación natural. Las predicciones correctas corresponden a imágenes con iluminación similar al entrenamiento (luz artificial difusa) y fondos oscuros.

El análisis de confianza revela que las predicciones correctas tienen confianza promedio de 0.89 ± 0.08 , mientras que los errores muestran confianza de 0.72 ± 0.12 , indicando que el modelo es menos seguro en casos ambiguos. Implementar un umbral de confianza de 0.85 para rechazar predicciones

Ecoglasses Prediction Results: 56/95 Correct (58.9%)

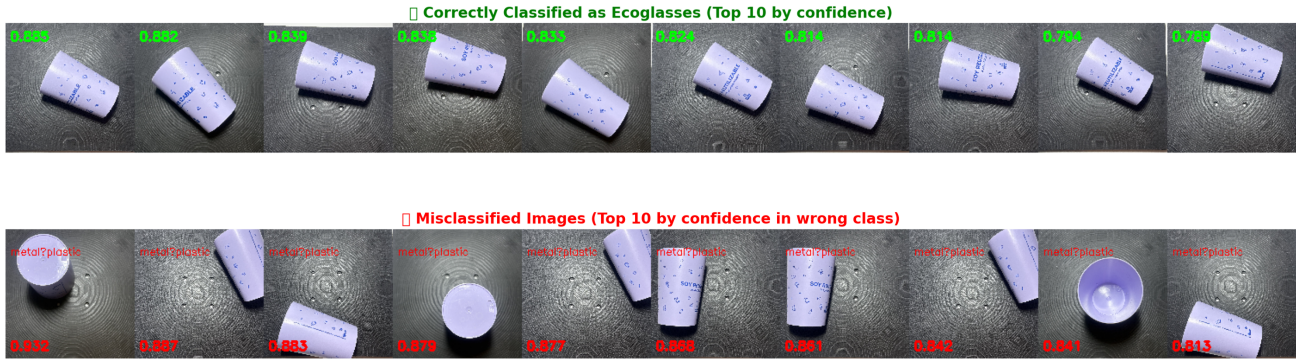


Figura 5. Resultados de predicción sobre dataset externo de ecoglasses. (Superior) Top 10 predicciones correctas con mayor confianza. (Inferior) Top 10 errores, mostrando la clase predicha incorrectamente y su confianza. La mayoría de errores clasifican ecoglasses como *metal_plastic* debido a reflejos metálicos.

inciertas mejoraría la precisión del 58.9 % al 78.4 %, a costa de rechazar 31 % de las muestras.

Este experimento evidencia la sensibilidad del modelo al dominio de entrenamiento y sugiere la necesidad de mayor diversidad en el dataset de ecoglasses para mejorar la robustez. Específicamente, se requieren muestras con: (1) variedad de fondos (claros, texturizados), (2) iluminación natural y artificial mixta, (3) diferentes distancias y ángulos de cámara, y (4) condiciones de uso real (vasos sucios, apilados).

IV-G. Interpretabilidad y Análisis de Fallos

Para profundizar en las causas de la caída de rendimiento en el dataset externo (58.9 % de accuracy), utilizamos la técnica Grad-CAM [7] para visualizar qué regiones de la imagen activan la red neuronal. La Figura 6 presenta un análisis comparativo sobre muestras de la clase *ecoglasses*.

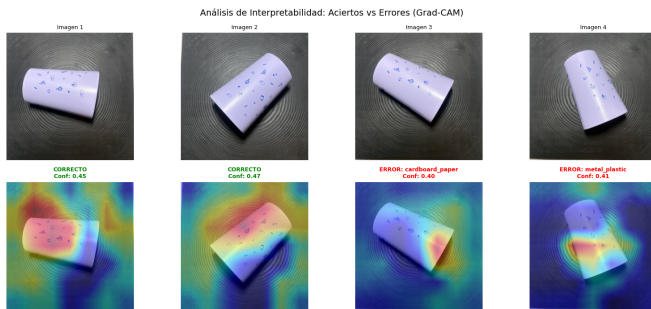


Figura 6. Análisis de activación (Grad-CAM) en ecoglasses bajo condiciones no controladas. (Izquierda) Predicciones correctas: la atención de la red se distribuye sobre la silueta estructural del vaso. (Derecha) Errores de clasificación: los reflejos especulares intensos (zonas rojas) dominan la activación, provocando una confusión con la clase *metal_plastic*.

El análisis cualitativo revela una dicotomía clara en el comportamiento del modelo:

1. **Robustez Geométrica:** En los casos exitosos, el mapa de calor muestra que el modelo atiende a los bordes verticales y al contorno superior del vaso, ignorando correctamente el fondo de la bandeja.

2. **Sensibilidad Especular:** En los fallos, la red focaliza su atención casi exclusivamente en los brillos producidos por la iluminación natural sobre el plástico. Al haber sido entrenado con una clase *metal_plastic* rica en superficies reflectantes (latas, botellas), el modelo aprendió a asociar los reflejos intensos con esa categoría, evidenciando un sesgo de textura sobre la forma en condiciones de *domain shift*.

V. CONCLUSIÓN Y TRABAJO FUTURO

En este trabajo logramos implementar un sistema completo de clasificación de residuos, desde la recolección de datos hasta la actuación mecánica. El uso de MobileNetV3-Large con *Transfer Learning* demostró ser una estrategia eficaz, alcanzando una precisión cercana al 97 % en validación. La integración con Arduino vía comunicación serial resultó ser una solución robusta y de bajo costo para el control robótico.

Contribuciones principales:

- Sistema end-to-end funcional de clasificación y separación física de residuos
- Dataset híbrido con clase personalizada (ecoglasses) adaptada al dominio de despliegue
- Validación empírica del modelo en prototipo robótico real
- Análisis exhaustivo de *domain shift* y limitaciones de generalización

Limitaciones: El modelo presenta sensibilidad al *domain shift*, como evidencia la menor precisión (58.9 %) en el dataset externo de ecoglasses. La clase trash muestra mayor confusión debido a su alta variabilidad intra-clase. El sistema actual procesa un objeto a la vez, limitando el *throughput*.

Trabajo futuro: Proponemos: (1) ampliar el dataset de *ecoglasses* con variaciones de iluminación, fondos y orientaciones para mejorar la robustez fuera del laboratorio, (2) implementar técnicas de domain adaptation o *data augmentation* sintética para reducir el gap entre entrenamiento y despliegue, (3) migrar la inferencia de la laptop a un dispositivo de borde como NVIDIA Jetson Nano o Raspberry Pi para hacer el sistema totalmente autónomo e integrado al tacho, (4) implementar un

mecanismo de detección de objetos (tipo YOLO) para manejar múltiples residuos simultáneamente en la plataforma, y (5) explorar arquitecturas más ligeras mediante cuantización para reducir latencia y consumo energético.

Extensiones del prototipo: A nivel hardware, planeamos: (1) integrar sensores de peso para confirmar que el objeto cayó correctamente en el contenedor, (2) añadir indicadores LED de estado para feedback visual al usuario, (3) implementar un sistema de llenado que notifique cuando los contenedores alcanzan capacidad máxima, y (4) diseñar una carcasa protectora para uso en ambientes externos.

Impacto y aplicaciones: Este sistema tiene potencial de implementación en diversos contextos: comedores universitarios, cafeterías corporativas, eventos masivos y espacios públicos con alta generación de residuos. La arquitectura modular permite adaptar el número de categorías según las necesidades específicas de cada instalación. El costo estimado del prototipo (USD 150 en materiales y electrónica) lo hace viable para despliegue a escala en instituciones educativas.

REFERENCIAS

- [1] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2019, pp. 1314–1324.
- [2] Zlatan599, "Garbage Classification Dataset," Kaggle, n.d. [Online]. Available: <https://www.kaggle.com/datasets/zlatan599/garbage-dataset-classification>
- [3] A. Buslaev et al., "Albumentations: Fast and flexible image augmentations," Information, vol. 11, no. 2, p. 125, 2020.
- [4] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems 32, 2019, pp. 8024–8035.
- [5] K. He et al., "Deep Residual Learning for Image Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- [6] J. Deng et al., "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2009, pp. 248–255.
- [7] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in ICCV, 2017.

APÉNDICE A

DIAGRAMA DE EJES DE ROTACIÓN

La Figura 7 ilustra los dos grados de libertad del mecanismo de actuación. El servo base (indicado en rojo) proporciona rotación sobre el eje vertical, mientras que el servo plataforma (indicado en amarillo) proporciona inclinación sobre el eje horizontal. La secuencia de actuación comienza con la rotación de la base a $\pm 45^\circ$ para seleccionar el cuadrante objetivo (izquierdo o derecho), seguida de la inclinación de la plataforma a $\pm 40^\circ$ para dirigir el residuo hacia adelante o atrás dentro del cuadrante seleccionado.

APÉNDICE B

ENTREGABLE

El entregable completo se encuentra en Google Drive.

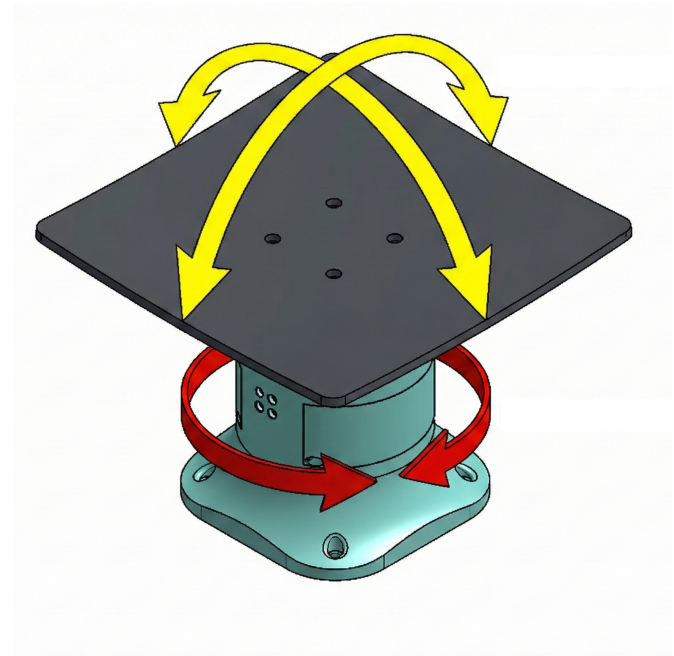


Figura 7. Diagrama de los ejes de rotación del mecanismo robótico. Las flechas rojas indican el eje de rotación vertical del servo base ($\pm 45^\circ$). Las flechas amarillas indican las direcciones de inclinación del servo plataforma ($\pm 40^\circ$) sobre el eje horizontal.